# Assignment 1: Build the Model from Scratch

March 19, 2026

## Overview

In Assignment 1, we contain **5 questions** to explore the usage and configuration of supervised learning models. We then apply these concepts to a real-world classification task, carrying out data exploration, preprocessing, and model evaluation techniques.

## Instructions

Please submit your solution by **April 9, 11:59 PM.**
**Late submissions are not accepted.** You can use any third-party libraries or AI tools to assist your work, **but if plagiarism or cheating on assignments, the grade will be 0.**

**Submissions consist of two parts:**

1. **Code**
    a. Please host your **code on a public GitHub repository** (commit before deadline).
2. **Report.**
    a. Please ensure your public **GitHub link is included** within your report, and submit your **report via E3.**
    b. **File name:** DM_asg1_{studentID}.pdf

Please write down your report in English. The TA will test whether your code can run successfully.

**Github Repository:** [https://github.com/NYCU-Data-Mining/DM2026-Assignment-1](https://github.com/NYCU-Data-Mining/DM2026-Assignment-1)

---

## 1. Linear Regression (20 points):

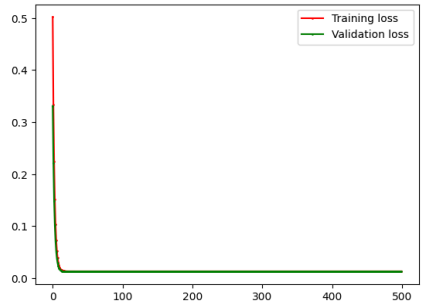
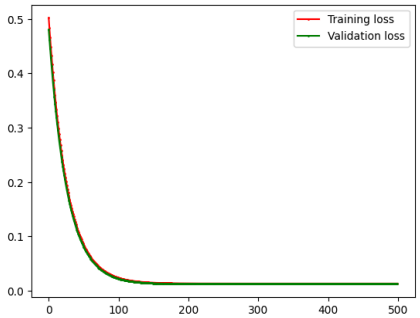
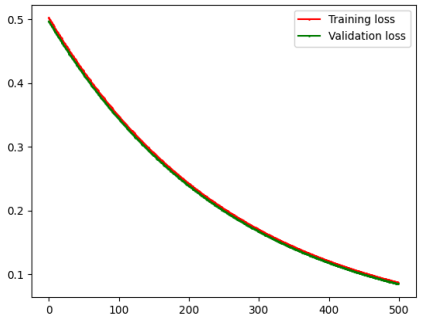
Please refer to **Linear_Regression.ipynb** and file(**metrics.py**) in the model folder. **Do not change random_seed**, as it will affect the evaluation results.

Apply the following questions 1(a) to 1(c) to **all datasets** (linear_data_A, linear_data_B, linear_data_C, and linear_data_D) in the data/linear_data folder.

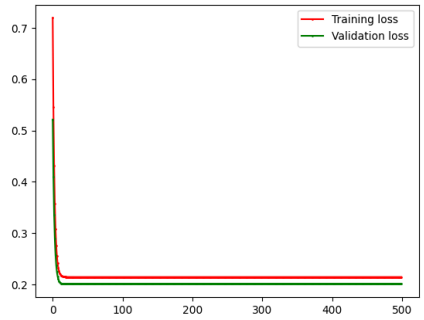
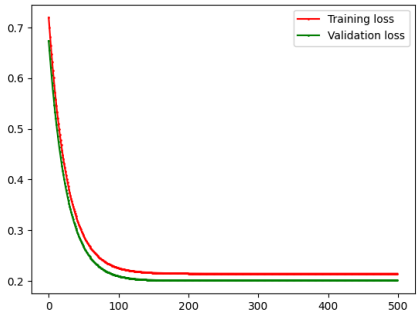
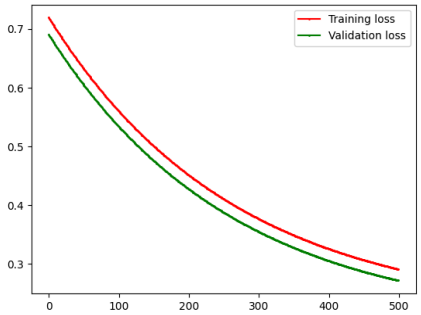
a. **Change the learning rate to the following values, and plot the loss curve for each learning rate:**

$$\text{learning rate} \in \{0.1, 0.01, 0.001\}$$

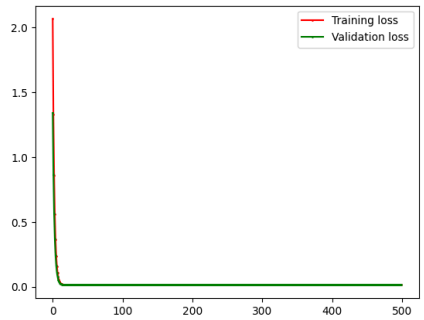
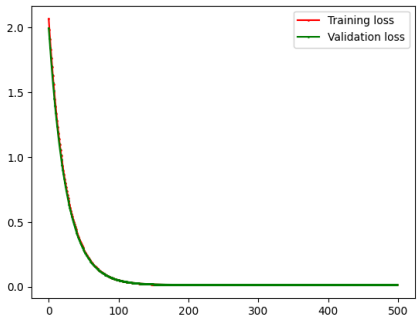
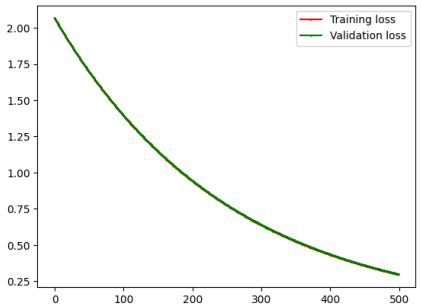
linear_data_A

| 0.1 | 0.01 | 0.001 |
|---|---|---|
|   |   |   |

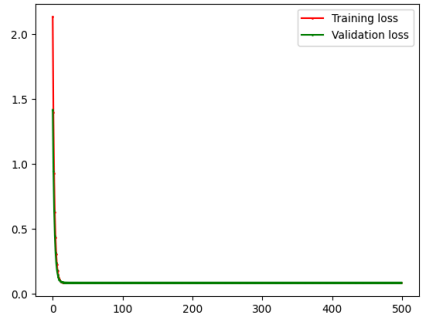
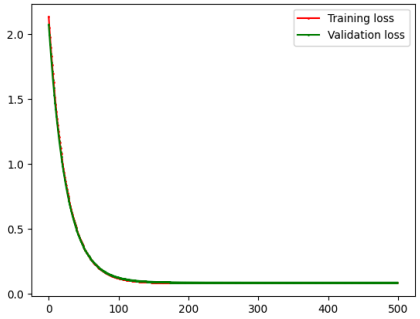
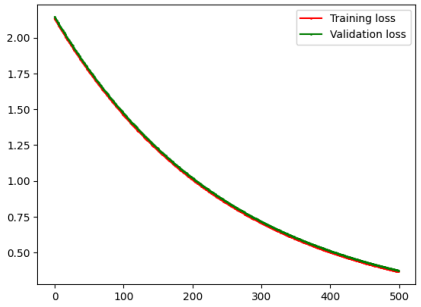
linear_data_B

| 0.1 | 0.01 | 0.001 |
|---|---|---|
|   |   |   |

linear_data_C

| 0.1 | 0.01 | 0.001 |
|---|---|---|
|   |   |   |

linear_data_D

| 0.1 | 0.01 | 0.001 |
|---|---|---|
|  |  |  |

**b. Following 1(a), show the evaluations by using the "evaluate_linear_regression" function:**

linear_data_A

| 0.1 | 0.01 | 0.001 |
|---|---|---|
| MSE : 0.0134<br>MAE : 0.1010<br>RMSE : 0.1156<br>R-squared : 0.5690 | MSE : 0.0134<br>MAE : 0.1010<br>RMSE : 0.1156<br>R-squared : 0.5690 | MSE : 0.0832<br>MAE : 0.2369<br>RMSE : 0.2884<br>R-squared : -1.6836 |

linear_data_B

| 0.1 | 0.01 | 0.001 |
|---|---|---|
| MSE : 0.2137<br>MAE : 0.4038<br>RMSE : 0.4623<br>R-squared : 0.0448 | MSE : 0.2137<br>MAE : 0.4038<br>RMSE : 0.4623<br>R-squared : 0.0448 | MSE : 0.2687<br>MAE : 0.4404<br>RMSE : 0.5184<br>R-squared : -0.2011 |

linear_data_C

| 0.1 | 0.01 | 0.001 |
|---|---|---|
| MSE : 0.0132<br>MAE : 0.0987<br>RMSE : 0.1148<br>R-squared : 0.9954 | MSE : 0.0132<br>MAE : 0.0987<br>RMSE : 0.1148<br>R-squared : 0.9954 | MSE : 0.2852<br>MAE : 0.4273<br>RMSE : 0.5341<br>R-squared : 0.8994 |

linear_data_D

| 0.1 | 0.01 | 0.001 |
|---|---|---|

| | | |
|---|---|---|
| MSE : 0.0824<br>MAE : 0.2467<br>RMSE : 0.2870<br>R-squared : 0.9716 | MSE : 0.0824<br>MAE : 0.2467<br>RMSE : 0.2870<br>R-squared : 0.9716 | MSE : 0.3540<br>MAE : 0.4778<br>RMSE : 0.5950<br>R-squared : 0.8780 |

### c. Describe your observations in 1(a) and 1(b).

We can see in 1(a) that the loss is the highest for linear_data_B (when we look at the y-axes, the smallest value is close to 0.2, while for the other examples it is close to 0). And indeed, the difference is confirmed by the 1(b) stats (MSE is used as the loss in the figures).

Plus, we notice that the higher the learning rate, the smaller the loss. This makes sense since the model learns faster with a higher learning rate; thus, it fits the training data faster, which leads to a smaller loss.

## 2. Logistic Regression (20 points):

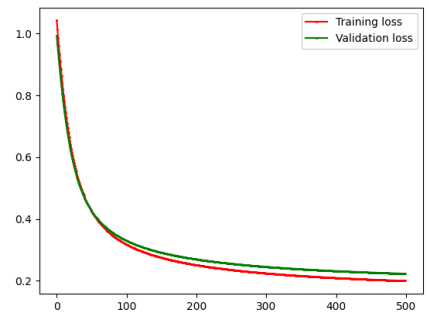
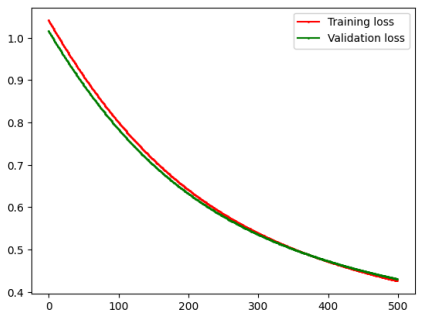
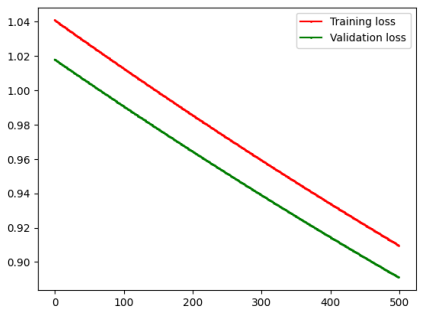
Please refer to **Logistic_Regression.ipynb** and file(**metrics.py**) in the model folder. **Do not change random_seed**, as it will affect the evaluation results.

Apply the following questions 2(a) to 2(f) to **all datasets** (logistic_data_A, logistic_data_B, logistic_data_C, and logistic_data_D) in the data/logistic_data folder.

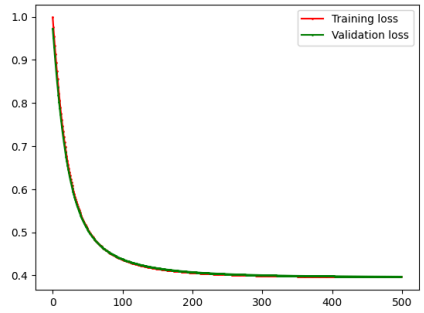
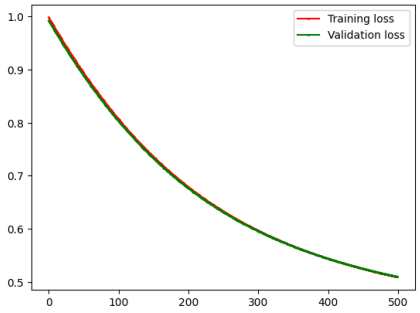
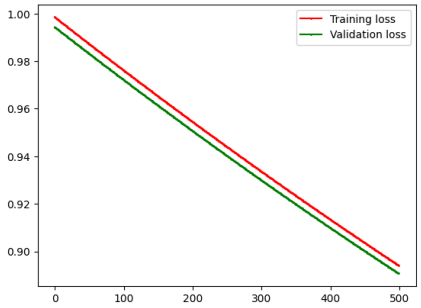
### a. Change the learning rate to the following values, and plot the loss curve for each learning rate:

$$\text{learning rate} \in \{0.1, 0.01, 0.001\}$$

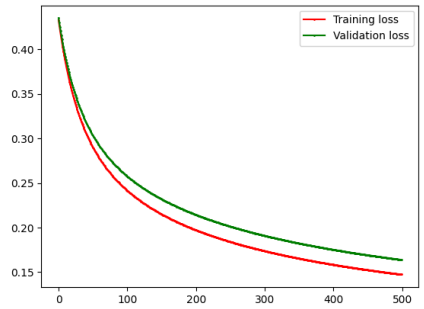
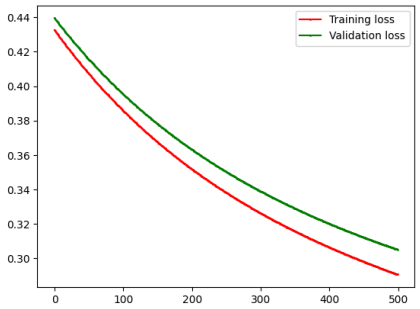
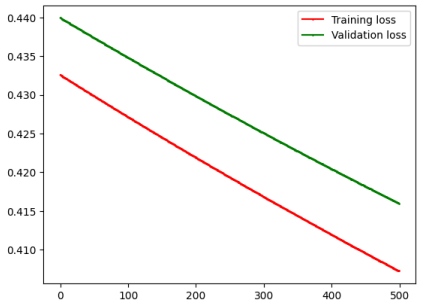
logistic_data_A

| 0.1 | 0.01 | 0.001 |
|---|---|---|
|  |  |  |

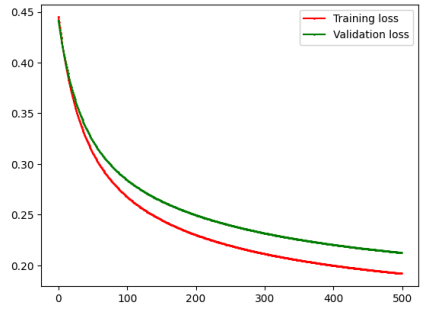
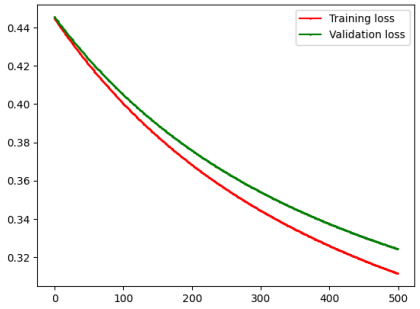
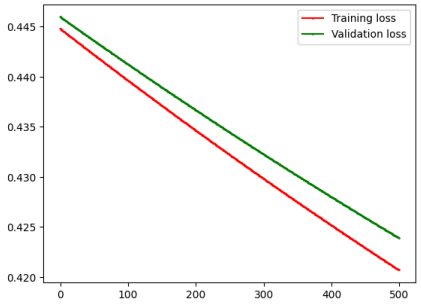
logistic_data_B

| 0.1 | 0.01 | 0.001 |
|---|---|---|
|   |   |   |

logistic_data_C

| 0.1 | 0.01 | 0.001 |
|---|---|---|
|   |   |   |

logistic_data_D

| 0.1 | 0.01 | 0.001 |
|---|---|---|
|   |   |   |

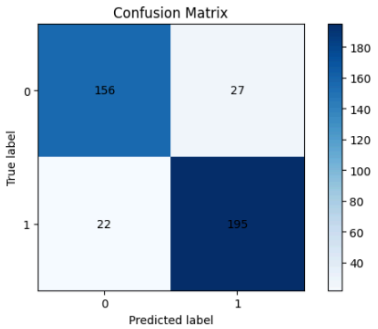
**b. Following 2(a), show the evaluations by using the "evaluate_binary_classifier" function.**

[Hint]

- ❖ Each dataset shows three evaluation results (learning rate∈{0.1, 0.01, 0.001}).
- ❖ Each evaluation (shown in a screenshot, as the figure on the left) contains the metrics: Accuracy, Precision, Recall, F1 score.
- ❖ Before evaluating, you must complete the TODO sections (as the figure on the right) in model/metrics.py. Please use the pre-imported sklearn package.

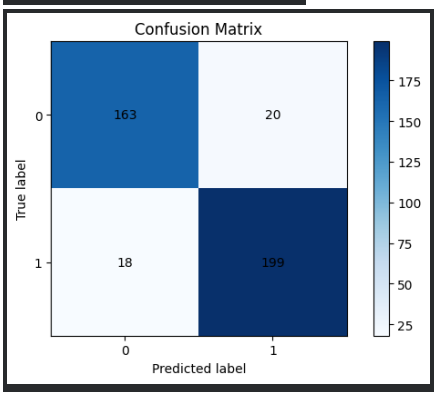
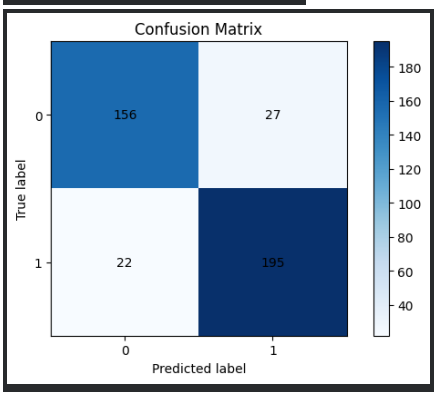
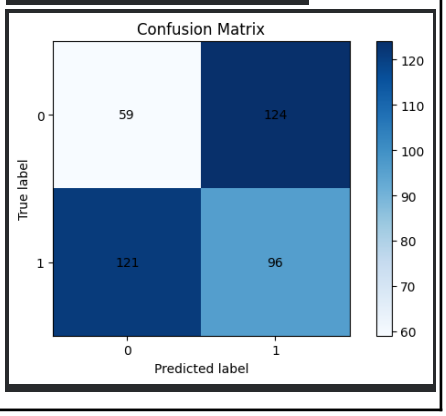
```
# Evaluation
# Please complete the evaluate_binary_classifier in the model/metrics.py by the sklearn package.
evaluate_binary_classifier(y_true=y_val,y_pred=y_val_pred,title='Logistic Regression Evaluation')
```

```
Logistic Regression Evaluation
Accuracy  : 0.8775
Precision : 0.8784
Recall    : 0.8986
F1-score  : 0.8884
```




```
# TODO
metrics = {
    'Accuracy': 'TODO: use sklearn.metrics to compute accuracy',
    'Precision': 'TODO: use sklearn.metrics to compute Precision',
    'Recall': 'TODO: use sklearn.metrics to compute Recall',
    'F1-score': 'TODO: use sklearn.metrics to compute F1-score'
}
```

logistic_data_A

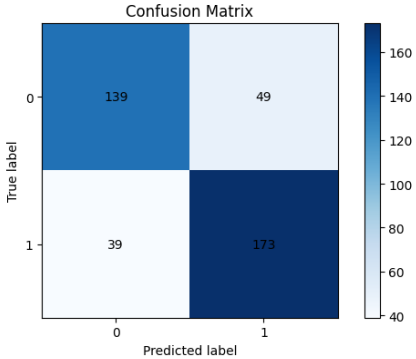
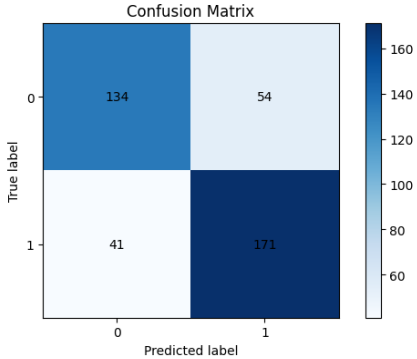
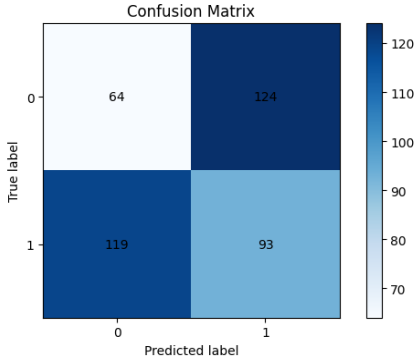
| 0.1 | 0.01 | 0.001 |
|---|---|---|
| Accuracy : 0.9050<br>Precision : 0.9087<br>Recall : 0.9171<br>F1-score : 0.9128 | Accuracy : 0.8775<br>Precision : 0.8784<br>Recall : 0.8986<br>F1-score : 0.8884 | Accuracy : 0.3875<br>Precision : 0.4364<br>Recall : 0.4424<br>F1-score : 0.4394 |
|  |  |  |



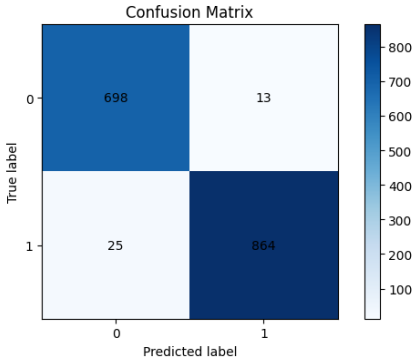
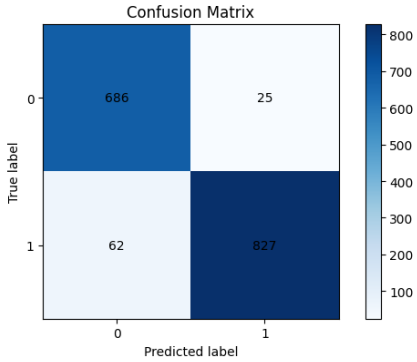
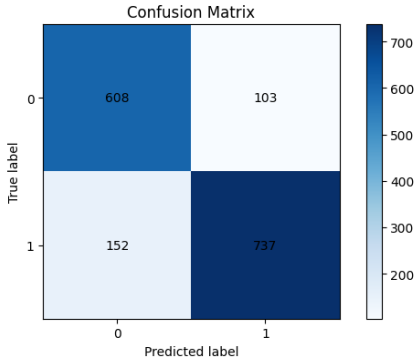




logistic_data_B

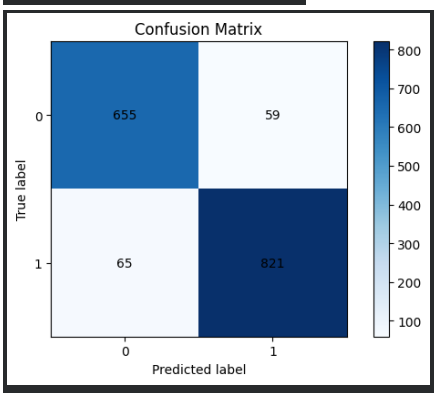
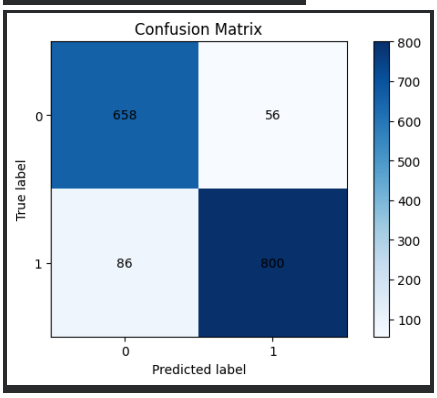
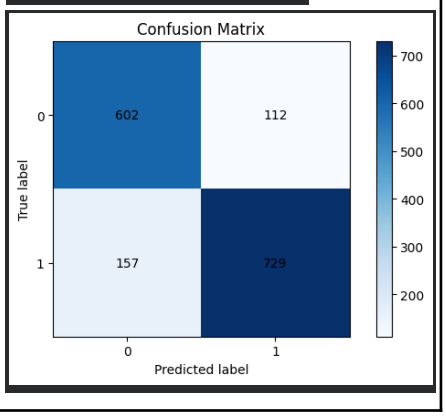
| 0.1 | 0.01 | 0.001 |
|---|---|---|
| Accuracy : 0.7800<br>Precision : 0.7793<br>Recall : 0.8160<br>F1-score : 0.7972<br>  | Accuracy : 0.7625<br>Precision : 0.7600<br>Recall : 0.8066<br>F1-score : 0.7826<br>  | Accuracy : 0.3925<br>Precision : 0.4286<br>Recall : 0.4387<br>F1-score : 0.4336<br>  |

logistic_data_C

| 0.1 | 0.01 | 0.001 |
|---|---|---|
| Accuracy : 0.9762<br>Precision : 0.9852<br>Recall : 0.9719<br>F1-score : 0.9785<br>  | Accuracy : 0.9456<br>Precision : 0.9707<br>Recall : 0.9303<br>F1-score : 0.9500<br>  | Accuracy : 0.8406<br>Precision : 0.8774<br>Recall : 0.8290<br>F1-score : 0.8525<br>  |

logistic_data_D

| 0.1 | 0.01 | 0.001 |
|---|---|---|
| Accuracy : 0.9225<br>Precision : 0.9330<br>Recall : 0.9266 | Accuracy : 0.9113<br>Precision : 0.9346<br>Recall : 0.9029 | Accuracy : 0.8319<br>Precision : 0.8668<br>Recall : 0.8228 |

| | | |
|---|---|---|
| F1-score : 0.9298   | F1-score : 0.9185   | F1-score : 0.8442   |

### c. Describe your observations in 2(a) and 2(b).

Similar to what I observed in 1(a) and 1(b):

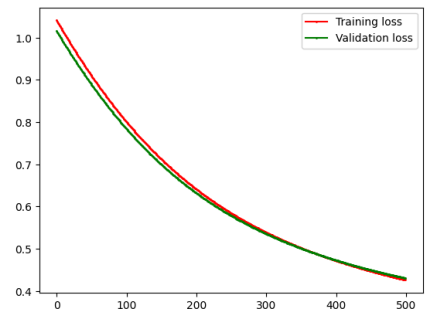
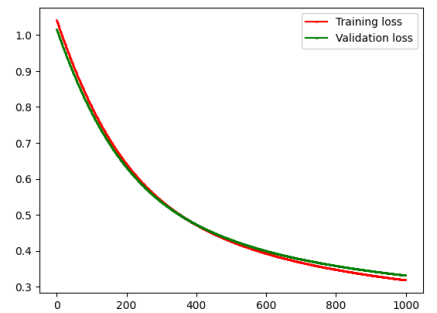
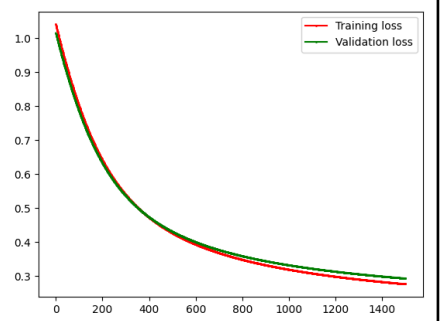
We can see in 2(a) that the loss is in general (for each learning rate) the highest for logistic_data_B. And indeed, the difference is confirmed by the 2(b) stats, since for logistic_data_B, the accuracy is lower.

Plus, we notice that the higher the learning rate, the better the precision. This makes sense since the model learns faster with a higher learning rate; thus, it fits the training data faster, which leads to greater accuracy.

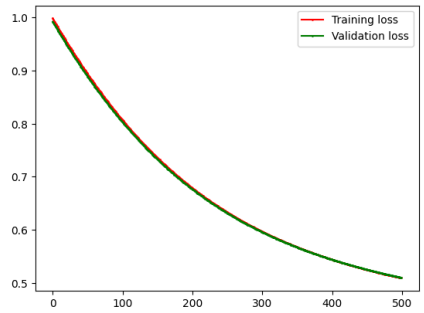
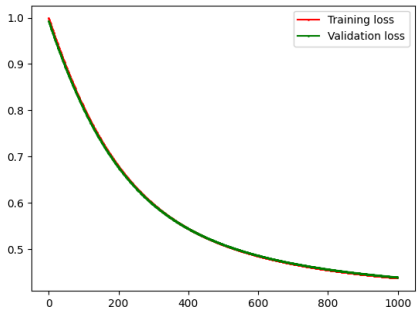
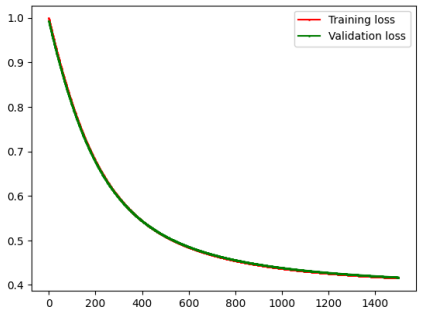
### d. Change the number of training iterations to the following values, and plot the loss curve for each setting (using a fixed learning rate of 0.01):

$$n_iteration \in \{500, 1000, 1500\}$$

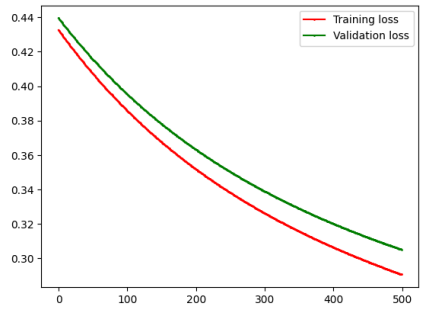
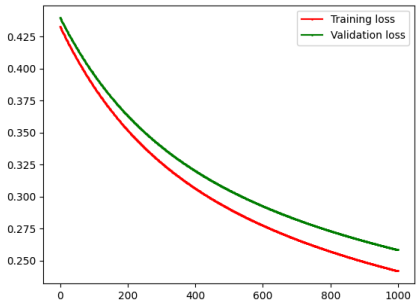
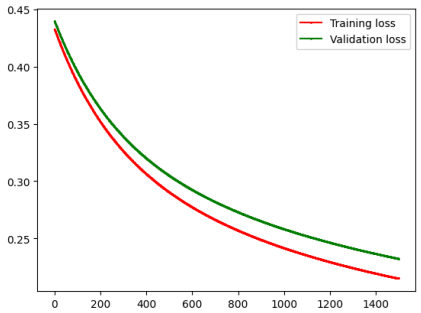
logistic_data_A

| 500 | 1000 | 1500 |
|---|---|---|
|   |   |   |

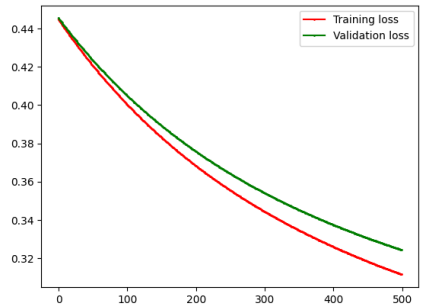
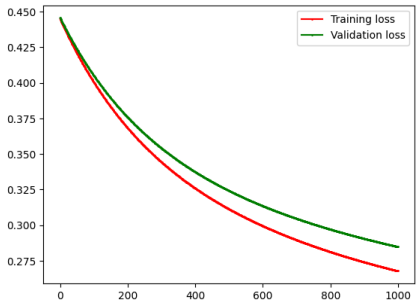
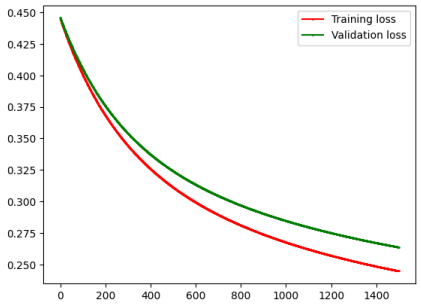
logistic_data_B

| 500 | 1000 | 1500 |
|---|---|---|
|   |   |   |

logistic_data_C

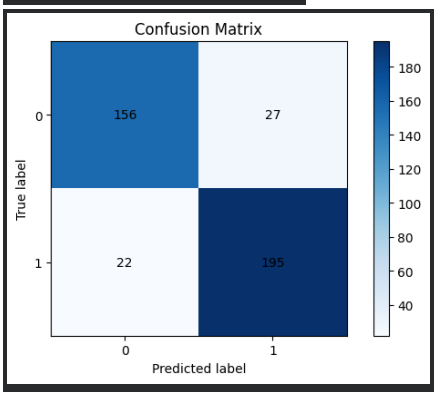
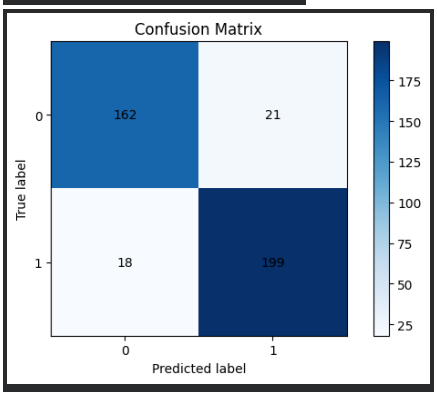
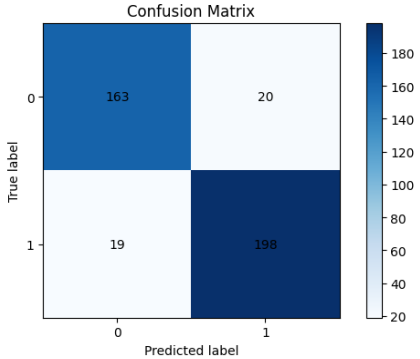
| 500 | 1000 | 1500 |
|---|---|---|
|   |   |   |

logistic_data_D

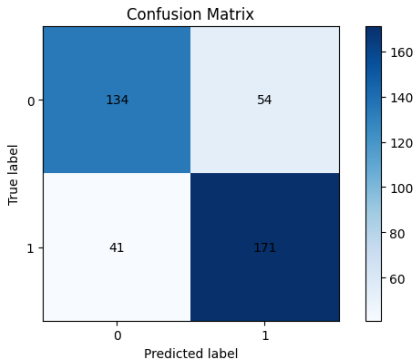
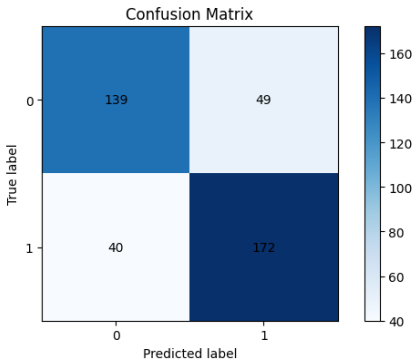
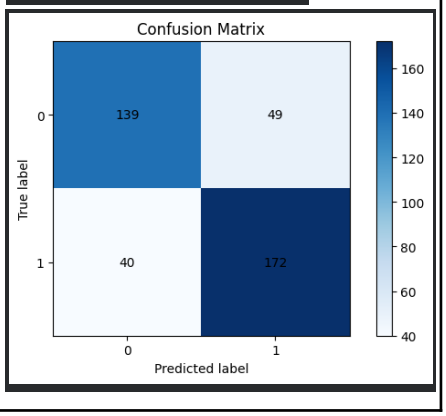
| 500 | 1000 | 1500 |
|---|---|---|
|   |   |   |

**e. Following 2(d), show the evaluations by using the "evaluate_binary_classifier" function.**

logistic_data_A

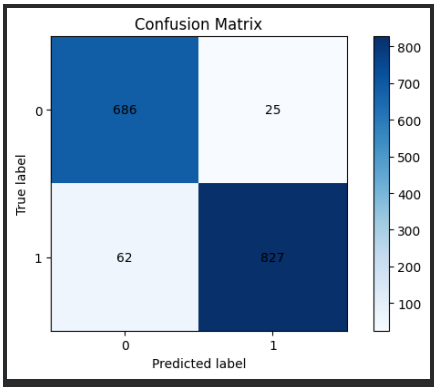
| 500 | 1000 | 1500 |
|---|---|---|
| Accuracy : 0.8775<br>Precision : 0.8784<br>Recall : 0.8986<br>F1-score : 0.8884<br>  | Accuracy : 0.9025<br>Precision : 0.9045<br>Recall : 0.9171<br>F1-score : 0.9108<br>  | Accuracy : 0.9025<br>Precision : 0.9083<br>Recall : 0.9124<br>F1-score : 0.9103<br>  |

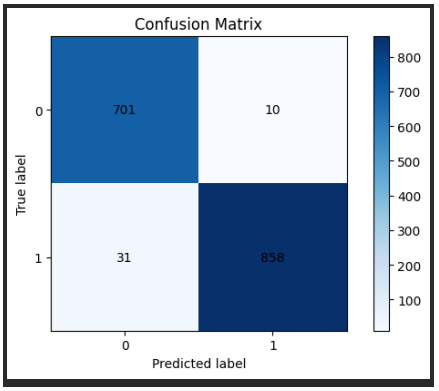
logistic_data_B

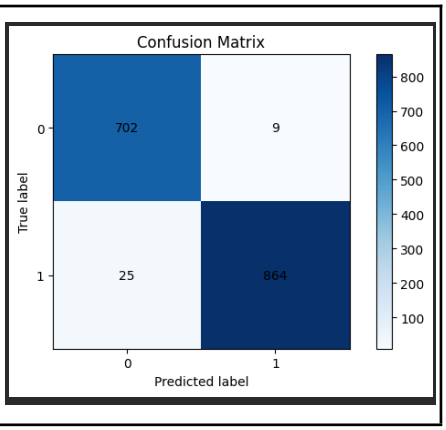
| 500 | 1000 | 1500 |
|---|---|---|
| Accuracy : 0.7625<br>Precision : 0.7600<br>Recall : 0.8066<br>F1-score : 0.7826<br>  | Accuracy : 0.7775<br>Precision : 0.7783<br>Recall : 0.8113<br>F1-score : 0.7945<br>  | Accuracy : 0.7775<br>Precision : 0.7783<br>Recall : 0.8113<br>F1-score : 0.7945<br>  |

logistic_data_C

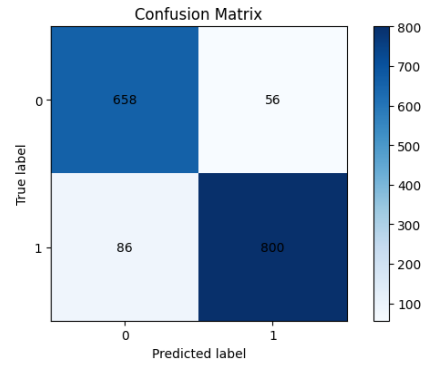
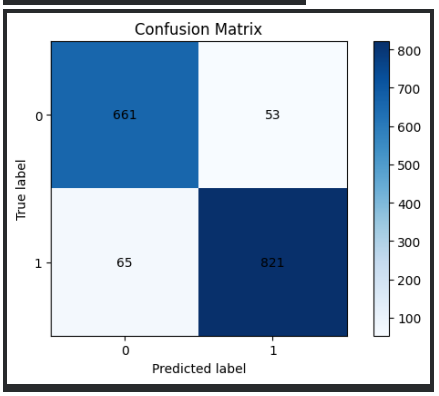
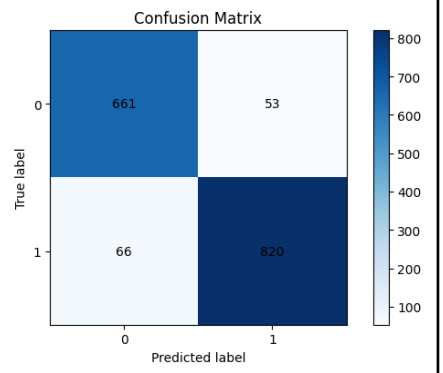
| 500 | 1000 | 1500 |
|---|---|---|
| Accuracy : 0.9456<br>Precision : 0.9707<br>Recall : 0.9303<br>F1-score : 0.9500 | Accuracy : 0.9744<br>Precision : 0.9885<br>Recall : 0.9651<br>F1-score : 0.9767 | Accuracy : 0.9788<br>Precision : 0.9897<br>Recall : 0.9719<br>F1-score : 0.9807 |




logistic_data_D

| 500 | 1000 | 1500 |
|---|---|---|
| Accuracy : 0.9113<br>Precision : 0.9346<br>Recall : 0.9029<br>F1-score : 0.9185 | Accuracy : 0.9263<br>Precision : 0.9394<br>Recall : 0.9266<br>F1-score : 0.9330 | Accuracy : 0.9256<br>Precision : 0.9393<br>Recall : 0.9255<br>F1-score : 0.9323 |
|  |  |  |







### f. Describe your observations in 2(d) and 2(e).

The greater the number of iterations, the better the result (lower loss and greater accuracy). This makes sense, since the more the model is trained, the better it will fit the data. Also, the training effect is best in the beginning; then, the model slowly converges.

## 3. Data Preprocessing (20 points):

Please refer to **Real_World_Classification.ipynb**. **Do not change random_seed**, as it will affect the evaluation results. NYCU_Iris.csv is in the data folder.

**a.** Report the **median** and **standard deviation** for columns with missing values.

**b.** Write the code to **replace the missing values** using "**Nearest Neighbors Imputation**" for columns with missing values. Report the **median** and **standard deviation** of those columns. Compare your result with 3(a) and **explain your observation**.

[Hint]

- ❖ sklearn.impute

```
# TODO: Replace the missing values using "Nearest Neighbors Imputation"
# ---------- Start your code below ----------


# ---------- -------------------- ----------
```

## 4. Data Exploration (20 points):

Please continue using **Real_World_Classification.ipynb**. Based on the preprocessed data from Task 3, proceed with the data exploration. Do not change the random_seed to ensure consistency in evaluation results.

**a.** **Plot the histogram** of the column "PetalWidthCm" in "NYCU_Iris.csv". Please **describe your observations** for the histogram.

**b.** **List the feature** (excluding "PetalWidthCm", "Species", and "Id") that has the **largest positive correlation** (Pearson correlation coefficient) **with the "PetalWidthCm"** column.

[Hint]

- ❖ Pearson correlation: sklearn.feature_selection.r_regression

**c.** **Report the top 5 features** with the **strongest negative** correlations with the 'PetalWidthCm' column.

**d.** **Plot the boxplot** for all features found in 4(b) and 4(c).

## 5. Regularization (20 points):

Please continue using **Real_World_Classification.ipynb**. Based on the preprocessed data from Task 3, proceed with the data exploration. Do not change the random_seed to ensure consistency in evaluation results.

a. **L2 Regularization.** Please run Real_World_Classification.ipynb to compare the performance between no regularization (current code) and L2 regularization. For L2 regularization, set the regularization parameter (reg_lambda in the code) to 0.01, 1, and 100, respectively. Then, **plot the loss curves** for all settings:

   (1) No regularization
   (2) L2 (reg_lambda=0.01)
   (3) L2 (reg_lambda=1)
   (4) L2 (reg_lambda=100)

   [Hint]

   - Four figures in total, presented in a format similar to that in 2(a)
   - To use L2 regularization, please set the reg_type='l2' and reg_lambda value in the fit() function. For example: model = LinearModel (..., reg_type='l2', reg_lambda=0.01)

b. **Following 5(a), show the testing results by using the "evaluate_binary_classifier" function.**

   [Hint]

   - Show two testing results (no regularization and L2).
   - Each testing result (shown in a screenshot) contains the metrics: Accuracy, Precision, Recall, F1 score, presented in a format similar to that in 2(b).

c. **Describe your observations in 5(a) and 5(b).**

# Reference


1. https://github.com/anhquan0412/basic_model_scratch/tree/master
2. https://scikit-learn.org/stable/api/sklearn.metrics.html
3. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/fork-a-repo


# FAQ

Below are some common questions and answers. If you have any other questions, please feel free to contact the TA at nycu.dm.ta@gmail.com.

**1. How to download the code from github?**

1. Fork the repository: Click the "Fork" button at the top right to create a copy under your own GitHub account.
2. Clone to your local machine:

   Open your terminal and run: git clone https://github.com/XXX/XXX.git

If this is your first time contributing to a project, you can follow this official tutorial: Fork a repo - GitHub Docs.

**2. How to upload the code to your github repo?**

Once you have finished making changes to your code locally, you can upload them to your GitHub repository by following these steps:

```shell
# If you used 'git clone', you can skip this part.
git init
git remote add origin <Your_Repository_URL>

# Stage and Commit
git add .
git commit -m "assignment1"

# Upload
git push origin main
```

If you need more details on managing files, you can refer to this guide: Adding a file to a repository - GitHub Docs